Implementação de Servidor Web com *Socket*

Matheus C. Damasceno, William de S. Gomes

[1]Centro de Engenharia, Modelagem e Ciências Sociais Aplicadas
Universidade Federal do ABC (UFABC)
Av. dos Estados, 5001 – Bangú – Santo André – SP

{matheus.costa, gomes.souza}@aluno.ufabc.edu.br

Resumo. *Sockets se referem a interfaces de software pelas quais um processo envia e recebe mensagens da rede. Neste projeto foi realizada a implementação de um servidor utilizando Python, capaz de atender a múltiplas requisições simultâneas de navegadores web, por meio de sockets. Para tal, foram implementadas funções capazes de estabelecer conexão com o socket do navegador, receber e analisar requisições, e também criadas páginas de conteúdo e erro. De acordo com a solicitação, o servidor cria uma resposta HTTP contendo a mensagem solicitada ou uma mensagem de erro, caso haja algum problema. Para verificar seu funcionamento, foram realizados testes utilizando diferentes sistemas operacionais para execução do servidor, diferentes sistemas e dispositivos clientes, além de testes utilizando a rede local e a internet.*

Abstract. *Sockets are software interfaces through which a process sends and receives messages from the network. In this project, a server was implemented using Python, capable of establishing many simultaneous requests from web browsers, through sockets. To this aim, functions were implemented to establish a connection with the browser socket, receive and analyze requests, and also it was created content and error pages. According to the request, the server creates an HTTP response containing the requested message or an error message, in case there is a problem. To check its correct functioning, tests were performed using different operating systems for running the server and different systems and devices for the client. In addition, tests were performed using both local and internet networks.*

1. Introdução

Quando se deseja estabelecer comunicação entre dois sistemas finais, as mensagens trocadas realizam um percurso pela rede através de sistemas físicos e os protocolos associados para realização de tal feito. Na camada de aplicação, um processo se comunica com outro a partir da leitura e escrita em seus respectivos *sockets*, os quais ficam responsáveis por encaminhar e receber as mensagens através da rede [Kurose and Ross 2013]. Assim, define-se então *socket* como um *endpoint* no *link* de comunicação entre processos separados.

Em um par de sistemas finais diferentes em uma aplicação, dizemos que um desses é o cliente, o qual realiza requisições, e o outro é o servidor, que pode responder tais solicitações caso uma conexão seja estabelecida entre eles, comumente referido processo

cliente/servidor, no qual um cliente faz uma requisição ao servidor podendo ou não estabelecer uma conexão e receber uma resposta. Para que a comunicação seja possível, é necessário que cada processo possua um identificador único na rede, o qual é estabelecido pelo par endereço de IP + porta. Uma vez que esse identificador é único e associado a cada processo, não há confusão no recebimento de múltiplas mensagens para aplicações diferentes sendo executadas em uma mesma máquina, tendo em vista que cada uma será direcionada para o seu respectivo *socket*.

2. Objetivos

Esse projeto teve como objetivo construir um servidor *web* usando *socket* e TCP/IP.

2.1. Objetivos específicos

1. Criação de *socket* baseado em TCP/IPv4;
2. Implementação de servidor *web* local;
3. Realização de requisições HTTP;
4. Implementação de mensagens de erro;
5. Criação de páginas HTML de acesso ao conteúdo;
6. Acesso ao servidor a partir de IP público.

3. Conceitos Relacionados

A internet, nos seus primórdios na década de 60, servia como ferramenta para universidades e militares para troca de arquivos e troca de informações, e ainda que em seus primeiros passos, estava muito ligada à pesquisa. Em 1989 Berners-Lee inventou a *World Wide Web* (WWW) [Berners-Lee et al. 1992], uma aplicação que permite o acesso de informações, vídeos, aplicativos, áudios, textos por um meio visual no navegador, tornando assim a internet conhecida e utilizada pelo mundo inteiro, sendo a *web* o seu principal uso. Por trás de tal invenção tem-se o HTTP, o protocolo que sustenta a *web* [Fielding et al. 1999]. O HTTP (*HiperText Transfer Protocol*) é executado no servidor e no cliente, que conversam entre si por meio de troca de mensagens.

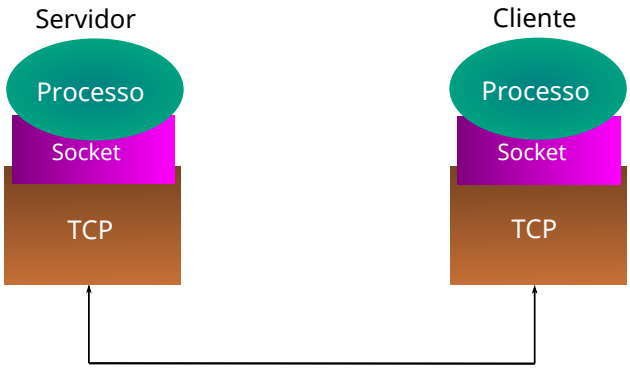

Figura 1. Diagrama comunicação entre cliente e servidor.

Como foi explicado, quando dois processos em sistemas finais desejam se comunicar, a mensagem é passada a um *socket*, que por sua vez a encaminha para a camada de

transporte, ficando esta responsável por garantir que a mensagem chegue a seu destino, no qual haverá um outro *socket* que receberá a mensagem e a encaminhará para aplicação, conforme esquematizado na Figura 1. Para o projeto foi escolhido o TCP (*Transfer Control Protocol*) como protocolo de transporte. O TCP se encarregará de dividir as mensagens em pacotes, anexar um cabeçalho a cada um deles e realizar o seu transporte e, por se tratar de um protocolo confiável, sabe-se que os pacotes alcançarão seu destino final.

4. Metodologia

A implementação do servidor foi realizada utilizado Python, versão 2, por meio da biblioteca nativa de *socket* presente na linguagem. A Figura 2 apresenta o esquemático dos arquivos presentes no diretório do servidor. O algoritmo para a execução do mesmo foi programado em um único arquivo, *WebServer.py*, que inclui as funções e estrutura básica para a execução do servidor.

Inicialmente, definiu-se um objeto *socket* com os parâmetros

1. AF_INET
2. SOCK_STREAM,

responsáveis por definir o endereçamento por IPv4 e o protocolo de envio de mensagens TCP. Na sequência, definiu-se a porta 6789 como padrão a ser utilizada. Apesar da porta 80 ser a padrão para *web*, optou-se por utilizar uma porta anônima por questões de segurança, uma vez que o projeto levou em consideração a abertura de portas de acesso externo no *firewall* do *gateway* para que fosse possível o acesso ao servidor por meio da internet.

Definidos os parâmetros iniciais, a execução contínua do servidor é mantida por um loop while, com condição de parada dada por meio de uma requisição externa, como será explicado posteriormente. A conexão com o cliente se dá por meio do método `accept()` e, para que fosse possível a conexão de múltiplos clientes simultaneamente, foi utilizada a biblioteca *thread*, que permitiu que cada conexão estabelecida fosse processada em uma *thread* diferente. Uma vez estabelecida a conexão, captura-se a requisição do cliente por meio do método `recv()` – que conterá a mensagem enviada pelo cliente – e a processa, de modo a obter a informação com o caminho do arquivo ou comando de requisição.

As solicitações válidas podem ser o caminho de um arquivo presente no diretório raiz ou nas subpastas do servidor, ou então o simples acesso ao direto ao endereço e porta do servidor, que o fará retornar a página inicial *index.html*. Também é possível que o cliente requisite um endereço inválido, mas nesse caso, o algoritmo o considerará como erro 404. Em cada um dos casos, o servidor abre o arquivo requisitado, que pode ser uma página de acesso ou de erro, ou mesmo um documento ou arquivo de mídia, prepara um cabeçalho HTTP com o código de status e envia ambos para o cliente utilizando o método `send()`.

Dada a imprevisibilidade da execução final do sistema e levando em consideração que espera-se que um servidor esteja sempre disponível, também foi considerado um tratamento especial e, caso haja algum erro inesperado no servidor, este retorna a página de erro 500 para a solicitação, continuando em execução normalmente. Além disso, também foi implementada uma função que permite encerrar o servidor adequadamente de forma remota, bastando que o cliente requisite o endereço /kill.

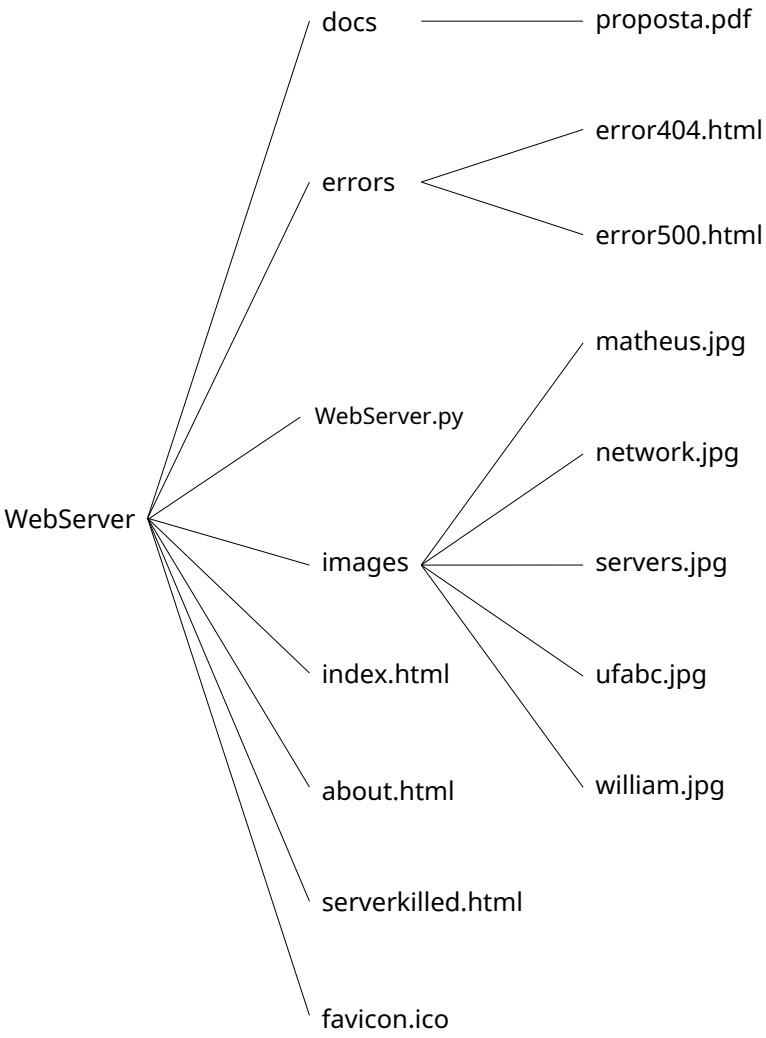

Figura 2. Diretório do servidor.

5. Teste e Avaliação

Com o script do servidor *web* pronto, executou-se o seguinte comando no terminal do Linux, no diretório raiz do projeto, para inicialização do processo:

```
$ python2 WebServer.py
```

Com o processo do servidor em execução, utilizou-se o navegador Google Chrome para acessar o endereço e a porta do servidor. Vale ressaltar que neste caso, não foi necessária a implementação de um *script* de *socket* para o cliente, tendo em vista que o próprio navegador já realiza este papel. Com isso, observou-se a seguinte mensagem no exibida no terminal do servidor:

```
Connection established

Client Address:
    192.168.0.35:52012
```

```

Request:
    GET / HTTP/1.1
Host:
    192.168.0.34:6789
Connection:
    keep-alive
User-Agent:
    Mozilla/5.0 (Windows NT 6.1; Win64; x64)
    AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/83.0.4103.61
    Safari/537.36
Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,
    image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
Accept-Encoding:
    gzip, deflate
Accept-Language:
    pt-BR,pt;q=0.9,en-US;q=0.8,en;q=0.7

Sending 'index.html' to 192.168.0.35:52011
```

a qual apresenta informações de endereço do cliente e servidor, bem como as mensagem de requisição e o arquivo enviado como resposta. Além disso, observou-se que a primeira solicitação era referente apenas ao aquivo index.html da página inicial, e que após este ser transferido, o navegador automaticamente requisitou outros elementos ligados a ele, tais como as figuras e o ícone da página. Também foi possível verificar a capacidade do servidor de responder não apenas páginas html, como também arquivos pdf, vídeos, imagens e outros conteúdo de mídia. Todo esse processo foi realizado também no sistema operacional Windows e os testes apresentaram os mesmos resultados.

Já no cliente, o navegador exibiu corretamente a página inicial presente no diretório do servidor, conforme observado na Figura 3. Ao clicar nos hiperlinks presentes na página, é possível navegar por elas normalmente, no qual, cada clique corresponde à requisição de uma nova página e seus respectivos conteúdos. Apesar de os testes realizados terem sido feitos com páginas simples, observou-se durante testes preliminares que, enquanto a conexão estava ativa, o servidor ficava inacessível para outros clientes. Isso motivou a implementação da função *multi-thread* no servidor, e como isso, caso houvessem requisições simultâneas por mais de um cliente, ambos receberão a resposta do servidor, já que é criado uma nova *thread* para cada um.

Por fim, para um teste completo do servidor, foram realizados testes de conexão utilizando a internet. Para tal, foram adicionadas regras de redirecionamento de porta no roteador da rede doméstica ao qual o servidor estava conectado, o que permitiu que este fosse acessado a partir requisições externas. Os testes foram realizados com o servidor sendo executado em uma rede doméstica em Carapicuíba/SP e requisições a partir de dois lugares distintos: uma rede doméstica em São Paulo/SP e uma rede corporativa em Santo André/SP. Os testes foram bem sucedidos e apresentaram os mesmos resultados que os executados em rede local.

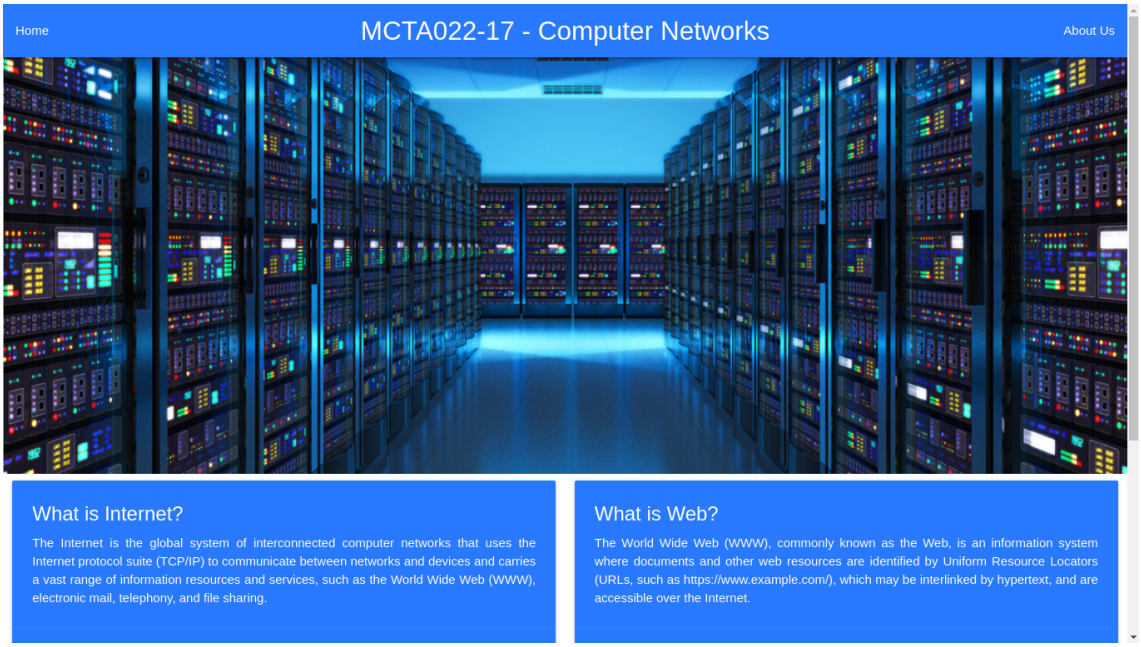

Figura 3. Página inicial requisitada pelo cliente.

6. Conclusão

Com a realização deste projeto, foi possível implementar com êxito um servidor capaz se ser executado em ambientes Linux e Windows e apto a atender a múltiplas requisições simultaneamente a partir de navegadores *web*. O servidor apresentou funcionamento adequado com clientes *desktop* e *mobile* e o acesso às páginas HTML e arquivos de mídia foram realizados com sucesso. Além disso, ao realizar requisições e comandos inválidos, o servidor continuou sendo executado normalmente, enviando as respectivas mensagens de erro implementadas para o cliente.

A partir dos testes executados, verificou-se que o servidor foi capaz de atender não somente requisições feitas a partir da rede local, como também por meio da internet, funcionando adequadamente em ambos os casos.

Em projetos futuros, espera-se poder realizar a implementação de funções extras que permitam a utilização de outros comandos para a interação entre cliente e servidor, como por exemplo, na integração com bancos de dados, além de recursos extras de segurança, tais como a utilização de TSL e HTTPS para que seja possível criar conexões criptografadas entre o cliente e servidor.

Referências

Berners-Lee, T., Cailliau, R., Groff, J.-F., and Pollermann, B. (1992). World-wide web: the information universe. *Internet Research*.

Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., and Berners-Lee, T. (1999). Hypertext transfer protocol–http/1.1.

Kurose, J. and Ross, K. (2013). *Redes de computadores e a internet: uma abordagem top-down*. Pearson.